

Today's lecture is not in scope for the midterm.

- But it is needed for HW7 (after midterm).



Lecture 10

Subtype Polymorphism, Comparators, Comparables

CS61B, Fall 2025 @ UC Berkeley

Josh Hug and Peyrin Kao

Polymorphism vs. Function Passing

Lecture 10, CS61B, Fall 2025

Polymorphism vs. Function Passing

Comparable and Comparators

- Comparables
- Compilation Error Puzzle
- Comparators
- Comparables vs. Comparators

Summary

Today we're going to dig deeper into inheritance.

- We'll see the contrast between **polymorphism** and **function passing**.
- Unlike Python, where function passing is common, most idiomatic Java code relies more heavily on polymorphism.

Let's see some examples in Python.

Polymorphism Example in Python

```
class Dog:
    def __init__(self, name, size):
        self.name = name
        self.size = size

    def __gt__(self, other):
        return self.size > other.size
```

```
def get_the_max(x):
    max_value = x[0]
    for item in x:
        if item > max_value:
            max_value = item
    return max_value

max_dog = get_the_max(doglist)
```

In the code above, the **Dog** class overrides the > operator.

- Operator overloading is a form of **polymorphism**.
- Polymorphism: “the ability in programming to present the same programming interface for differing underlying forms” [[wiki](#)]
- Idea: We can use the > operator to compare anything (i.e. “any underlying form”) in Python, including Dogs.
 - Example: get_the_max uses >, which is specified by __gt__

Function Passing Example in Python

```
class Dog:
    def __init__(self, name, size):
        self.name = name
        self.size = size
    def __gt__(self, other):
        return self.size > other.size

def name_len(dog):
    return len(dog.name)
```

```
def get_the_max(x, key):
    max_value = x[0]
    for item in x:
        if key(item) > key(max_value):
            max_value = item
    return max_value

max_dog=get_the_max(doglist, name_len)
```

In the code above, the **get_the_max** function accepts a key function.

- This is an example of **function passing**.

Note: Strings have their own definition for **>**, so we also see here polymorphism.

Today we're going to dig deeper into inheritance.

- We'll see the contrast between **polymorphism** and **function passing**.
- Unlike Python, where function passing is common, most idiomatic Java code relies more heavily on polymorphism.

We'll spend a lot of time today writing these two examples in Java.

- Along the way, we'll see how the static typing system in Java makes this a bit trickier to express.

Comparables

Lecture 10, CS61B, Fall 2025

Polymorphism vs. Function Passing

Comparable and Comparators

- **Comparables**
 - Compilation Error Puzzle
 - Comparators
 - Comparables vs. Comparators

Summary

Dogs

Consider the dogs below.

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Grigometh", 200));  
dogs.add(new Dog("Pelusa", 5));  
dogs.add(new Dog("Clifford", 9000));
```



"Grigometh", size: 200



"Pelusa", size: 5



"Clifford", size: 9000

Dogs

To find the maximum, we can use `Collections.max`. Let's try this.

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Grigometh", 200));  
dogs.add(new Dog("Pelusa", 5));  
dogs.add(new Dog("Clifford", 9000));  
Dog maxDog = Collections.max(dogs);
```



“Grigometh”, size: 200



“Pelusa”, size: 5



“Clifford”, size: 9000

Dogs

To find the maximum, we can use `Collections.max`. Let's try this.

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Grigometh", 200));  
dogs.add(new Dog("Pelusa", 5));  
dogs.add(new Dog("Clifford", 9000));  
Dog maxDog = Collections.max(dogs);
```

This results in the incomprehensible error message below.

- The issue is that `max` doesn't know how to compare two `Dog` objects.

Required type: Collection <? extends T>

Provided: List <Dog>

reason: no instance(s) of type variable(s) T exist so that Dog conforms to Comparable<? super T>

Unlike Python, where we'd define a `__gt__` method to overload the `>` operator, in Java, we have to specify that a `Dog` is something that can be compared.

Question: How do we specify that a class has a certain capability in Java?

Unlike Python, where we'd define a `__gt__` method to overload the `>` operator, in Java, we have to specify that a `Dog` is something that can be compared.

Question: How do we specify that a class has a certain capability in Java?

- We must implement some interface.

The Comparable Interface

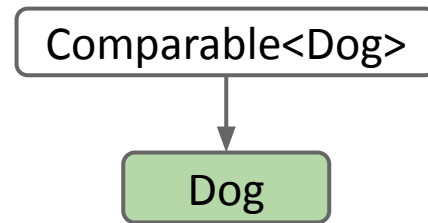
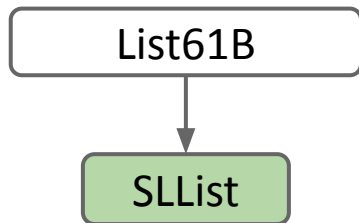
Unlike Python, where we'd define a `__gt__` method to overload the `>` operator, in Java, we have to specify that a `Dog` is something that can be compared.

In Java, we will declare to the world that a `Dog` is-a `Comparable`.

- Just like how earlier we said an `SLList` is-a `List61B`.

Since almost all of Java is written in Java, we can go look at `Comparable.java`.

- Link on github: [Comparable.java](#)



The Comparable Interface

Looking at [Comparable.java](#), we see:

```
public interface Comparable<T> {  
    /**  
     * Compares this object with the specified object for order.  
     * Returns a negative integer, zero, or a positive integer  
     * as this object is less than, equal to, or greater than the  
     * specified object.  
     * ...  
     */  
    public int compareTo(T o);  
}
```

Note: The full file is 142 lines long, almost all of which is documentation. This is not uncommon with really important parts of the library. Documentation is important.

Implementing Comparable

Let's implement the Comparable interface in Dog.java. This means:

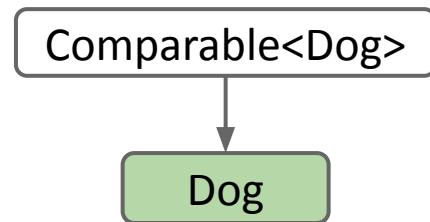
- Adding “implements Comparable<Dog>” to the top of the file.
- Overriding the compareTo method.

Implementing Comparable

Let's implement the Comparable interface in Dog.java. This means:

- Adding “implements Comparable<Dog>” to the top of the file.
- Overriding the compareTo method.

```
public class Dog implements Comparable<Dog> {  
    ...  
    @Override  
    public int compareTo(Dog uddaDog) {  
        if (size > uddaDog.size) {  
            return 1;  
        }  
        if (size < uddaDog.size) {  
            return -1;  
        }  
        return 0;  
    }  
}
```



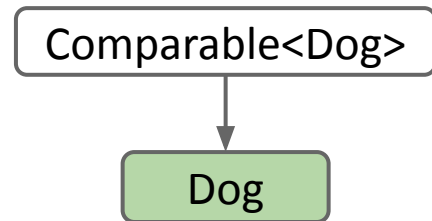
“uddaDog” courtesy of Harry Alu of [Ehu & Kai Adventures](#) (on the Big Island).

Implementing Comparable (Better Approach)

A cleaner implementation is shown below.

- This sort of code is very common in Java.

```
public class Dog implements Comparable<Dog> {  
    ...  
    @Override  
    public int compareTo(Dog uddaDog) {  
        return size - other.size;  
    }  
}
```



Now that we've implemented Comparable, the code below works fine!

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Grigometh", 200));  
dogs.add(new Dog("Pelusa", 5));  
dogs.add(new Dog("Clifford", 9000));  
Dog maxDog = Collections.max(dogs);
```

The flavor of polymorphism we've just employed is sometimes called **subtype polymorphism**.

- A supertype (Comparable) specifies the capability (in this case, comparison).
- A **subtype overrides** the supertype's abstract method.
 - Java decides what to do at runtime based on the type of the object that is invoking the method.

This is very different than Python, where there is a common `>` operator which can be overloaded.

Other Examples of Subtype Polymorphism

Subtype polymorphism is a very common idiom in Java.

Other examples:

- The Reader interface:
 - Keyboard: Gets input from the keyboard.
 - File: Reads
- A supertype (Comparable) specifies the capability (in this case, comparison).
- A **subtype overrides** the supertype's abstract method.
 - Java decides what to do at runtime based on the type of the object that is invoking the method.

This is very different than Python, where there is a common > operator which can be overloaded.

Compilation Error Puzzle

Lecture 10, CS61B, Fall 2025

Polymorphism vs. Function Passing

Comparable and Comparators

- Comparables
- **Compilation Error Puzzle**
- Comparators
- Comparables vs. Comparators

Summary

```
public class DogLauncher {  
    public static void main(String[] args) {  
        ...  
        Dog[] dogs = new Dog[]{d1, d2, d3};  
        System.out.println(Collections.max(dogs));  
    }  
}
```

```
public class Dog  
implements Comparable<Dog> {  
    ...  
    public int compareTo(Dog o) {  
        return this.size  
            - o.size;  
    } ...  
}
```

Q: If we omit `compareTo()`, which file will fail to **compile**?

- A. DogLauncher.java
- B. Dog.java
- C. Collections.java
- D. Comparable.java



```
public interface Collections {  
    // the max function  
    ...  
    int cmp = items[i].  
        compareTo(items[maxDex]);  
    ...  
} ...  
}
```

```
public class DogLauncher {  
    public static void main(String[] args) {  
        ...  
        Dog[] dogs = new Dog[]{d1, d2, d3};  
        System.out.println(Collections.max(dogs));  
    }  
}
```

```
public class Dog  
implements Comparable<Dog> {  
    ...  
    public int compareTo(Dog o) {  
        return this.size  
            - o.size;  
    } ...
```

Q: If we omit `implements Comparable<Dog>`, which file will fail to **compile**?

- A. DogLauncher.java
- B. Dog.java
- C. Collections.java
- D. Comparable.java



```
public interface Collections {  
    // the max function  
    ...  
    int cmp = items[i].  
        compareTo(items[maxDex]);  
    ...  
}...
```


Problem 1: Dog will fail to compile because it does not implement all abstract methods required by Comparable interface. (And I suppose DogLauncher will fail as well since Dog.class doesn't exist)

Problem 2: DogLauncher will fail, because it tries to pass things that are not Comparable, and Collections expects Comparables.

- If we used Override, Dog will not compile, because we have an override tag but we're not overriding (if we omit implements Comparable).

Comparators

Lecture 10, CS61B, Fall 2025

Polymorphism vs. Function Passing

Comparable and Comparators

- Comparables
- Compilation Error Puzzle
- **Comparators**
- Comparables vs. Comparators

Summary

The term “Natural Order” is sometimes used to refer to the ordering implied by a Comparable’s compareTo method.

- Example: Dog objects (as we’ve defined them) have a natural order given by their size.



“Pelusa”, size: 5



“Grigometh”, size: 200



“Clifford”, size: 9000

Alternate Sorting

May wish to order objects in a different way.

- Example: By Name.



“Clifford”, size: 9000



“Grigometh”, size: 200



“Pelusa”, size: 5

Reminder: Key Functions in Python

In Python, we achieved this goal using **function passing**.

```
class Dog:
    def __init__(self, name, size):
        self.name = name
        self.size = size
    def __gt__(self, other):
        return self.size > other.size

def name_len(dog):
    return len(dog.name)
```

```
def get_the_max(x, key):
    max_value = x[0]
    for item in x:
        if key(item) > key(max_value):
            max_value = item
    return max_value

max_dog=get_the_max(doglist, name_len)
```

By contrast, in Java, idiomatic code also uses **subtype polymorphism**.

The Comparator Interface

Java provides a Comparator interface for objects that are designed for comparing other objects.

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
    ...  
}
```

Comparator<Dog>

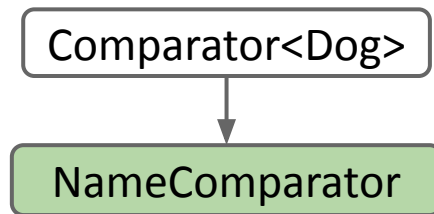
Let's try building and using one for comparing Dogs based on name.

Note: The Comparator interface has a LOT of default methods. We won't talk about them.

A Name Comparator

Java provides a `Comparator` interface for objects that are designed for comparing other objects.

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
    ...  
}
```



```
public static class NameComparator implements Comparator<Dog> {  
    @Override  
    public int compare(Dog a, Dog b) {  
        return a.name.compareTo(b.name);  
    }  
}
```

Comparator Quiz

What will be the output of the code below?

```
Dog a = new Dog("Frank", 1);  
Dog b = new Dog("Zeke", 1);  
Comparator<Dog> nc = new Dog.NameComparator();  
System.out.println(nc.compare(a, b));
```

```
public static class NameComparator implements Comparator<Dog> {  
    @Override  
    public int compare(Dog a, Dog b) {  
        return a.name.compareTo(b.name);  
    }  
}
```

- A. Positive number B. Negative number C. Zero

Comparator Quiz

What will be the output of the code below?

```
Dog a = new Dog("Frank", 1);  
Dog b = new Dog("Zeke", 1);  
Comparator<Dog> nc = new Dog.NameComparator();  
System.out.println(nc.compare(a, b));
```

```
public static class NameComparator implements Comparator<Dog> {  
    @Override  
    public int compare(Dog a, Dog b) {  
        return a.name.compareTo(b.name);  
    }  
}
```

The result will be a negative number: "Frank" is alphabetically less than "Zeke".

Using a Comparator to Find the Maximum

We can use our name comparator to find the dog with the maximum name.

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Grigometh", 200));  
dogs.add(new Dog("Pelusa", 5));  
dogs.add(new Dog("Clifford", 9000));  
Dog maxNameDog = Collections.max(dogs, new Dog.NameComparator());
```

Compare with Python below (using length of name rather than name itself).

```
def length_of_name(dog):  
    return len(dog.name)  
dogs = [Dog("Grigometh", 10),  
        Dog("Pelusa", 5),  
        Dog("Clifford", 9000)]  
max_dog = get_the_max(dogs, length_of_name)
```

This second argument
is an object of type
Comparator<Dog>

Using a Comparator to Find the Maximum

We can use our name comparator to find the dog with the maximum name.

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Grigometh", 200));  
dogs.add(new Dog("Pelusa", 5));  
dogs.add(new Dog("Clifford", 9000));  
Dog maxNameDog = Collections.max(dogs, new Dog.NameComparator());
```

Compare with Python below.

```
def length_of_name(dog):  
    return len(dog.name)  
dogs = [Dog("Grigometh", 10),  
        Dog("Pelusa", 5),  
        Dog("Clifford", 9000)]  
max_dog = get_the_max(dogs, length_of_name)
```

In Java we package our comparison function inside of a Comparator object. We rely on **subtype polymorphism**.

In Python we often **pass functions** directly.

Awkward Instantiation

The NameComparator instantiation is awkward and aesthetically unpleasant (to me).

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Grigometh", 200));  
dogs.add(new Dog("Pelusa", 5));  
dogs.add(new Dog("Clifford", 9000));  
Dog maxNameDog = Collections.max(dogs, new Dog.NameComparator());
```

One fix is to add a static variable reference to a pre-instantiated NameComparator.

Awkward Instantiation

The NameComparator instantiation is awkward and aesthetically unpleasant (to me).

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Grigometh", 200));  
dogs.add(new Dog("Pelusa", 5));  
dogs.add(new Dog("Clifford", 9000));  
Dog maxNameDog = Collections.max(dogs, new Dog.NameComparator());
```

One fix is to add a static variable reference to a pre-instantiated NameComparator. The usual naming convention is all caps:

```
public static NameComparator NAME_COMPARATOR = new NameComparator();
```

```
Dog maxNameDog = Collections.max(dogs, Dog.NAME_COMPARATOR);
```

A more modern approach is to use a lambda to define the Comparator.

- The lambda below defines and instantiates a Comparator object that compares the name of two dogs.
- We will not teach Java lambdas in 61B, nor will you be expected to learn them.

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Grigometh", 200));  
dogs.add(new Dog("Pelusa", 5));  
dogs.add(new Dog("Clifford", 9000));  
  
Comparator<Dog> dc = (d1, d2) -> d1.name.compareTo(d2.name);  
Dog maxNameDog = Collections.max(dogs, dc);
```

Comparables vs. Comparators

Lecture 10, CS61B, Fall 2025

Polymorphism vs. Function Passing

Comparable and Comparators

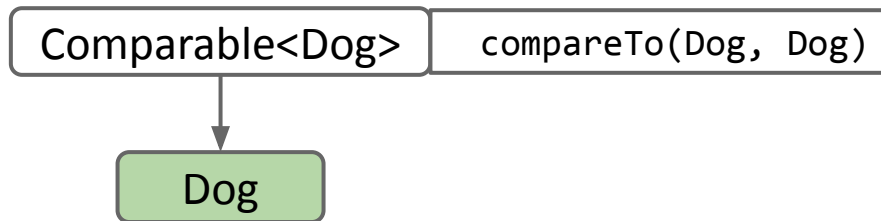
- Comparables
- Compilation Error Puzzle
- Comparators
- **Comparables vs. Comparators**

Summary

Comparables vs. Comparators

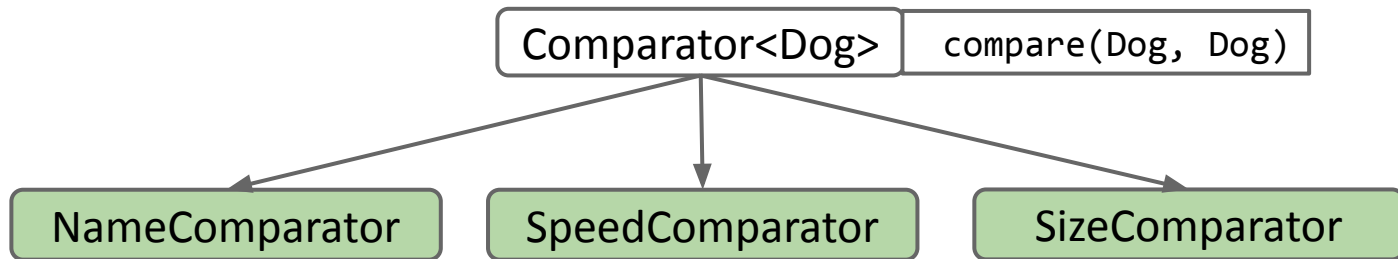
The Comparable interface specifies that a “natural order” exists.

- Instances of the class can compare themselves to other objects. Only one such order is possible.



The Comparator interface is used to compare extrinsically (by other classes).

- May have many such classes, each specifying one such order.



Summary

Lecture 10, CS61B, Fall 2025

Polymorphism vs. Function Passing
Comparable and Comparators

- Comparables
- Compilation Error Puzzle
- Comparators
- Comparables vs. Comparators

Summary

Summary

Today we've seen **polymorphism** and **function passing**.

Comparable<T>

T

For comparing two objects using an intrinsic order:

- Python uses a form of polymorphism called **operator overloading**.
- Java instead uses **subtype polymorphism**.

Python is duck typed: Do not have to specify if > is available or not.

```
class Dog:
    def __init__(self, name, size):
        self.name = name
        self.size = size

    def __gt__(self, other):
        return self.size > other.size
```

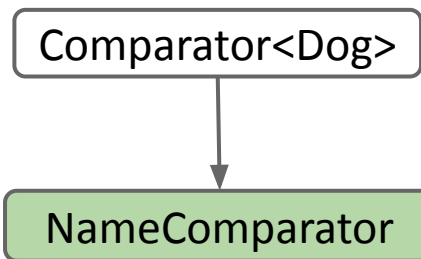
```
public class Dog implements Comparable<Dog> {
    ...
    @Override
    public int compareTo(Dog uddaDog) {
        return size - other.size;
    }
}
```

Summary

Today we've seen **polymorphism** and **function passing**.

For comparing two objects using an alternate order:

- Python uses **function passing**. You provide a key function.
- Java instead (usually) uses **subtype polymorphism**, just like for intrinsic orders. We package the comparison function in a “Comparator” object.



```
class Dog:
    def __init__(self, name, size):
        self.name = name
        self.size = size

def name_len(dog):
    return len(dog.name)

max_dog=max(doglist, key=name_len)
```

```
public static class NameComparator
    implements Comparator<Dog> {
    @Override
    public int compare(Dog a, Dog b) {
        return a.name.compareTo(b.name);
    }
}
```

Now For Something Completely Different

Lecture 10, CS61B, Fall 2025

The exam is entirely writing code.

- There are no “what does this weird code do?” type problems.
- There’s nothing fundamentally new on the exam. Not much in the way of “clever tricks”.

Important:

- Finish project 1 and 2.
- Complete discussion worksheet problems.

Can get extra practice on old exams.

- Focus on code writing problems.

Groups of people are WAY smarter than individual people.

Best way to study:

- Every day: Work on at least one interesting problem on your own. Don't look at the solutions. If you get stuck, you're stuck.
- Every day: Come back to problems you were stuck on before and see if you have new ideas.
- At least once: Get together with a group of other students and discuss your solutions (or attempts)
 - Best approach: Get up on a whiteboard or similar and write your solution. Let people interrupt to call out your good ideas or suggest improvements to your less good ideas. Metacognition is vital to top quality learning IMO.

Only look at solutions when you feel totally done with a problem. Unlikely to learn nearly as much as from the metacognition of discussing with others.

Attendance link: www.yellkey.com/idea

To-do:

- Project 2, due September 22.

